SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

ASSIGNMENT No : 6

Subject: Programming with Java

Name: Aman S. Choudhary

PRN: 25070122033

Div: CSE A1

Batch:2025-29

TITLE : Write a Java program to demonstrate the use of an inner class, an anonymous class.

Problem Statement

Write a Java program to demonstrate the use of an inner class, an anonymous class.

Learning Objectives

To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

Theory

An inner class is a class defined inside another class and has access to all members of the enclosing class, including private members. Inner classes help improve code organization and logical grouping of related classes. Anonymous classes are unnamed inner classes that are instantiated only once and are mainly used for implementing interfaces or abstract classes. They simplify code by eliminating the need for separate class definitions. These concepts are widely used in Java GUI programming and event handling

---

Outcome

The Java program successfully demonstrated the use of an inner class to access members of the enclosing class and an anonymous class to provide an on-the-fly implementation of a method without creating a separate named class. Both concepts executed correctly, producing the expected output.

Conclusion

Inner classes help in logically grouping classes that are only used in one place, improving encapsulation, while anonymous classes reduce code verbosity when only a single instance of a class is required. This experiment reinforced how nested and unnamed classes support cleaner, more modular Java code.

Exercise

1.Create a **Vehicle program** where an inner class displays vehicle details and anonymous class performs an action.

```java
class Vehicle {
    String name = "Car";

    class Details {
        void show() {
            System.out.println("Vehicle Name: " + name);
        }
    }
}
interface Action {
    void perform();
}
public class VehicleDemo {
    Run | Debug
    public static void main(String[] args) {
        Vehicle v = new Vehicle();
        Vehicle.Details d = v.new Details();
        d.show();

        Action a = new Action() {
            public void perform() {
                System.out.println(x: "Vehicle is moving...");
            }
        };
        a.perform();
    }
}
```

```
Vehicle Name: Car
Vehicle is moving...
```

---

2. Develop a **Food Delivery Application** where an inner class handles order details and anonymous classes handle delivery status updates.

```java
class Order {
    String item = "Pizza";

    class OrderDetails {
        void show() {
            System.out.println("Order Item: " + item);
        }
    }
}
interface DeliveryStatus {
    void update();
}
public class FoodDeliveryDemo {
    Run | Debug
    public static void main(String[] args) {
        Order o = new Order();
        Order.OrderDetails od = o.new OrderDetails();
        od.show();

        DeliveryStatus status = new DeliveryStatus() {
            public void update() {
                System.out.println(x: "Order is Out for Delivery");
            }
        };
        status.update();
    }
}
```

```
Order Item: Pizza
Order is Out for Delivery
```
